

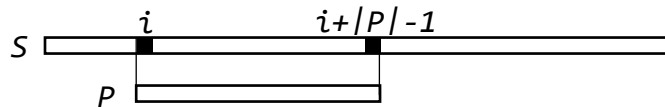
HÀM Z VÀ ỨNG DỤNG

1. Bài toán mở đầu

Cho 2 chuỗi $P(pattern)$ và $S(string)$.

- Bài toán 1: Tìm vị trí xuất hiện đầu tiên của P trong S
- Bài toán 2: Đếm số lần xuất hiện của P trong S , chỉ ra các vị trí xuất hiện lần lượt của P
- Bài toán 3: Viết liên tiếp một số lần P nhận được S . Cho S , hãy tìm P với độ dài nhỏ nhất. Ví dụ: $S = 'abababab' \Rightarrow P = 'ab'$

Với chiều dài của P và S nhỏ thì ta có thể giải quyết bằng thuật toán Brute Force



- Cho P trượt dần trên S và so sánh chuỗi $S[i..i + |P| - 1]$ và chuỗi P có trùng khớp nhau.
- Độ phức tạp của thao tác $O(|S| \times |P|)$

Khi chiều dài của 2 chuỗi lớn thì thuật toán KMP sẽ hoạt động hiệu quả hơn với độ phức tạp $O(|S| + |P|)$. Phần dưới đây sẽ giới thiệu thuật toán có độ phức tạp tương đương KMP nhưng cài đặt đơn giản hơn KMP.

2. Định nghĩa hàm Z

Cho chuỗi S có độ dài n ký tự. Hàm Z của chuỗi S là dãy số nguyên $z_0, z_1, \dots, z_{n-1}$, trong đó $z_i (0 \leq z_i < n)$ là độ dài phần đầu chung lớn nhất của 2 chuỗi $S[0..n-1]$ và $S[i..n-1]$

Ví dụ: với chuỗi $S = 'abaabab'$, ta có hàm $z = (0,0,1,3,0,2,0)$, quy ước $z_0 = 0$

i	$S[0..n-1]$	$S[i..n-1]$	Phần đầu chung dài nhất
0	abaabab	abaabab	' ' (quy ước)
1	abaabab	baabab	' '
2	abaabab	aabab	a
3	abaabab	abab	aba
4	abaabab	bab	' '
5	abaabab	ab	ab
6	abaabab	b	' '

Với dãy số nguyên có được sau khi tính xong hàm Z, xét $z_k = len(0 \leq k < n)$ cho ta biết

- Chuỗi gồm len kí tự đầu tiên của S sẽ xuất hiện lại ở vị trí k
- Số phần tử có giá trị len trong dãy Z cho biết số lần xuất hiện của len kí tự đầu tiên của S trong S .

Để tìm vị trí xuất hiện của chuỗi P trong chuỗi S ta ghép $S = P + \# + S$, với kí tự $\#$ là kí tự đặc biệt dùng để ngăn cách 2 chuỗi ($\#$ không xuất hiện trong chuỗi S). Sau đó tính hàm Z cho chuỗi S mới

Cài đặt hàm Z

```
for (int i = 1; i < n; i++)
{
    int j = 0, k = i;
    while (k < n && s[j] == s[k])
    {
        z[i]++;
        j++;
        k++;
    }
}
```

Độ phức tạp $O(n^2)$

Nhận xét:

- Giá trị tăng của biến j, k và $z[i]$ là cùng nhau, trong đó $j = z[i]$ (vì cùng được khởi tạo là 0) và $k = i + z[i]$ (do k được khởi tạo là i).
- Nói cách khác ta cần so sánh các cặp vị trí sau: $(s[0], s[i]), (s[1], s[i + 1]), (s[2], s[i + 2]), \dots, (s[k], s[i + k])$.

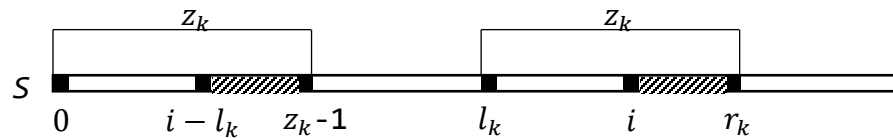
Đoạn lệnh được viết lại

```
for (int i = 1; i < n; i++)
    while (i + z[i] < n && s[z[i]] == s[i + z[i]])
        z[i]++;
```

Nhận xét

- Với mỗi vị trí $k < i$, chuỗi $S[0..n - 1]$ có đoạn đầu khớp dài nhất với chuỗi $S[k..n - 1]$ bắt đầu từ vị trí $l_k = k$ đến vị trí $r_k = k + z[k] - 1$.

- Gọi l_k, r_k là vị trí bắt đầu và kết thúc của đoạn khớp có đầu r_k lớn nhất trong số các vị trí k như thế.
- Khi tính giá trị hàm Z tại vị trí i , ta có 2 trường hợp sau:
 - + $i > r_k$: vị trí đang xét nằm ngoài vùng $[l_k, r_k]$ nên không thể tận dụng những gì đã tính được trước đó
 - + $i \leq r_k$: vị trí đang xét nằm trong đoạn khớp đã tìm được $[l_k, r_k]$. Hình minh họa cho thấy khi $i \leq r_k$ thì 2 chuỗi $s[i..r_k]$ và $s[i - l_k..z_k - 1]$ là giống nhau. Do đó ta có thể tận dụng giá trị hàm $z[i - l_k..z_k - 1]$ đã tính được để khởi tạo cho $z[i..r_k]$ thay vì phải khởi tạo $z[i..r_k] = 0$.



- + Lưu ý rằng $z[i] \leq z[i - l]$ vì chuỗi $s[i - l, z_k - 1]$ có thể còn được kéo dài. Giá trị khởi tạo của $z[i]$ không được vượt quá độ dài của phần cuối tính từ vị trí i đến r_k nên ta khởi tạo $z[i] = \min(z[i - l_k], r_k - i + 1)$
- + Khi tính xong $z[i]$ ta cập nhật lại đoạn $[l_k, r_k]$ nếu đầu r_k có thể kéo dài hơn nữa. Nghĩa là nếu $i + z[i] - 1 > r_k$ thì đoạn $[l_k, r_k]$ được cập nhật thành $[i, i + z[i] - 1]$

Đoạn chương trình cải tiến

```
int l = 0, r = 0;
for (int i = 1; i < n; i++)
{
    if (i <= r)
        z[i] = min(z[i-l], r-i+1);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]])
        z[i]++;
    if (i + z[i] - 1 > r)
    {
        l = i;
        r = i + z[i] - 1;
    }
}
```

Độ phức tạp cỡ $O(n)$

Bài tập ứng dụng:

Xét chuỗi S có chiều dài không quá 10^6 kí tự chữ cái latin in thường. Tiền tố độ dài k ($1 \leq k \leq n$) của S là một chuỗi gồm k kí tự đầu tiên của S .

Yêu cầu: Cho biết tiền tố xuất hiện nhiều nhất trong S . Nếu có nhiều kết quả thì đưa ra tiền tố có độ dài lớn nhất.